

```

package main

import (
    "encoding/json"
    "fmt"
    "io"
    "mime"
    "net"
    "net/http"
    "os"
    "path/filepath"
    "strings"
)

const PORT = 8000

var SHARE_FOLDER string

func getLocalIP() string {
    conn, err := net.Dial("udp", "8.8.8.8:80")
    if err != nil {
        return "127.0.0.1"
    }
    defer conn.Close()
    return conn.LocalAddr().(*net.UDPAddr).IP.String()
}

func isPathSafe(basePath, targetPath string) bool {
    absBase, _ := filepath.Abs(basePath)
    absTarget, _ := filepath.Abs(targetPath)
    return strings.HasPrefix(absTarget, absBase)
}

func listFilesHandler(w http.ResponseWriter, r *http.Request) {
    folderPath := strings.TrimPrefix(r.URL.Path, "/api/list")
    if folderPath == "" || folderPath == "/" {
        folderPath = SHARE_FOLDER
    } else {
        folderPath = filepath.Join(SHARE_FOLDER, strings.TrimPrefix(folderPath,
"/"))
    }

    if !isPathSafe(SHARE_FOLDER, folderPath) {
        http.Error(w, "Forbidden", http.StatusForbidden)
        return
    }

    info, err := os.Stat(folderPath)
    if err != nil || !info.IsDir() {
        http.Error(w, "Not Found", http.StatusNotFound)
        return
    }

    entries, err := os.ReadDir(folderPath)
    if err != nil {
        http.Error(w, "Error reading directory", http.StatusInternalServerError)
        return
    }

    // Initialize as empty array, not nil

```

```

items := make([]map[string]interface{}, 0)
for _, entry := range entries {
    info, _ := entry.Info()
    itemPath := filepath.Join(folderPath, entry.Name())
    relPath, _ := filepath.Rel(SHARE_FOLDER, itemPath)
    relPath = "/" + strings.ReplaceAll(relPath, "\\ ", "/")

    item := map[string]interface{}{
        "name": entry.Name(),
        "isDir": entry.IsDir(),
        "size": info.Size(),
        "path": relPath,
    }
    items = append(items, item)
}

w.Header().Set("Content-Type", "application/json")
json.NewEncoder(w).Encode(items)
}

func uploadFileHandler(w http.ResponseWriter, r *http.Request) {
    if r.Method != http.MethodPost {
        http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
        return
    }

    uploadPath := strings.TrimPrefix(r.URL.Path, "/api/upload")
    var targetFolder string
    if uploadPath == "" || uploadPath == "/" {
        targetFolder = SHARE_FOLDER
    } else {
        targetFolder = filepath.Join(SHARE_FOLDER, strings.TrimPrefix(uploadPath,
"/"))
    }

    if !isPathSafe(SHARE_FOLDER, targetFolder) {
        http.Error(w, "Forbidden", http.StatusForbidden)
        return
    }

    info, err := os.Stat(targetFolder)
    if err != nil || !info.IsDir() {
        http.Error(w, "Not Found", http.StatusNotFound)
        return
    }

    err = r.ParseMultipartForm(10 << 20)
    if err != nil {
        http.Error(w, "Bad Request", http.StatusBadRequest)
        return
    }

    // Initialize as empty array, not nil
    uploadedFiles := make([]map[string]interface{}, 0)

    for _, fileHeaders := range r.MultipartForm.File {
        for _, fileHeader := range fileHeaders {
            file, err := fileHeader.Open()
            if err != nil {
                continue
            }

```

```

        defer file.Close()

        filename := filepath.Base(fileHeader.Filename)
        if filename == "" {
            continue
        }

        filePath := filepath.Join(targetFolder, filename)
        if !isPathSafe(targetFolder, filePath) {
            continue
        }

        out, err := os.Create(filePath)
        if err != nil {
            continue
        }

        content, _ := io.ReadAll(file)
        out.Write(content)
        out.Close()

        uploadedFiles = append(uploadedFiles, map[string]interface{}{
            "name": filename,
            "size": len(content),
        })
    }
}

w.Header().Set("Content-Type", "application/json")
response := map[string]interface{}{
    "success": true,
    "files":   uploadedFiles,
}
json.NewEncoder(w).Encode(response)
}

func fileHandler(w http.ResponseWriter, r *http.Request) {
    if r.URL.Path == "/" || r.URL.Path == "" {
        w.Header().Set("Content-Type", "text/html")
        w.Write([]byte(getHTML()))
        return
    }

    filePath := filepath.Join(SHARE_FOLDER, strings.TrimPrefix(r.URL.Path, "/"))
    filePath, _ = filepath.Abs(filePath)

    if !isPathSafe(SHARE_FOLDER, filePath) {
        http.Error(w, "Forbidden", http.StatusForbidden)
        return
    }

    info, err := os.Stat(filePath)
    if err != nil || info.IsDir() {
        http.Error(w, "Not Found", http.StatusNotFound)
        return
    }

    mimeType := mime.TypeByExtension(filepath.Ext(filePath))
    if mimeType == "" {
        mimeType = "application/octet-stream"
    }
}

```

```

w.Header().Set("Content-Type", mimeType)
w.Header().Set("Content-Disposition", fmt.Sprintf("attachment; filename=%s",
filepath.Base(filePath)))

file, err := os.Open(filePath)
if err != nil {
    http.Error(w, "Internal Server Error", http.StatusInternalServerError)
    return
}
defer file.Close()

io.Copy(w, file)
}

func getHTML() string {
    return `<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>File Share</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            min-height: 100vh;
            display: flex;
            justify-content: center;
            align-items: center;
            padding: 20px;
        }
        .container {
            background: white;
            border-radius: 12px;
            box-shadow: 0 10px 40px rgba(0, 0, 0, 0.2);
            width: 100%;
            max-width: 900px;
            overflow: hidden;
        }
        .header {
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            padding: 30px;
            text-align: center;
        }
        .header h1 {
            font-size: 28px;
            margin-bottom: 8px;
        }
        .breadcrumb {
            background: #f5f5f5;
            padding: 12px 20px;
            font-size: 14px;
            color: #666;
            border-bottom: 1px solid #e0e0e0;

```

```

        display: flex;
        align-items: center;
        gap: 8px;
        flex-wrap: wrap;
    }
    .breadcrumb a {
        color: #667eea;
        text-decoration: none;
        cursor: pointer;
        padding: 4px 8px;
        border-radius: 4px;
        transition: background 0.2s;
    }
    .breadcrumb a:hover {
        background: #e8e8ff;
    }
    .content {
        padding: 20px;
        min-height: 300px;
    }
    .upload-area {
        border: 2px dashed #667eea;
        border-radius: 8px;
        padding: 30px;
        text-align: center;
        cursor: pointer;
        transition: all 0.3s;
        background: #f8f9ff;
        margin-bottom: 30px;
    }
    .upload-area:hover {
        background: #f0f0ff;
        border-color: #764ba2;
    }
    .upload-area.dragover {
        background: #e8e8ff;
        border-color: #764ba2;
        box-shadow: 0 4px 12px rgba(102, 126, 234, 0.3);
    }
    .upload-icon {
        font-size: 48px;
        margin-bottom: 15px;
    }
    .upload-text {
        font-size: 16px;
        color: #333;
        margin-bottom: 8px;
        font-weight: 500;
    }
    .upload-subtext {
        font-size: 13px;
        color: #999;
    }
    .file-grid {
        display: grid;
        grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
        gap: 15px;
        margin-bottom: 20px;
    }
    .file-item {
        background: #f9f9f9;

```

```

        border: 1px solid #e0e0e0;
        border-radius: 8px;
        padding: 15px;
        text-align: center;
        cursor: pointer;
        transition: all 0.3s;
        text-decoration: none;
        color: #333;
        position: relative;
    }
    .file-item:hover {
        background: #f0f0ff;
        border-color: #667eea;
        transform: translateY(-2px);
        box-shadow: 0 4px 12px rgba(102, 126, 234, 0.2);
    }
    .file-icon {
        font-size: 36px;
        margin-bottom: 10px;
    }
    .file-name {
        font-size: 13px;
        font-weight: 500;
        word-break: break-word;
    }
    .file-size {
        font-size: 11px;
        color: #999;
        margin-top: 8px;
    }
    .empty-state {
        text-align: center;
        padding: 60px 20px;
        color: #999;
    }
    .empty-state-icon {
        font-size: 64px;
        margin-bottom: 20px;
        opacity: 0.5;
    }
    .loader {
        text-align: center;
        padding: 40px 20px;
        color: #667eea;
    }
    .spinner {
        border: 4px solid #f0f0f0;
        border-top: 4px solid #667eea;
        border-radius: 50%;
        width: 40px;
        height: 40px;
        animation: spin 0.8s linear infinite;
        margin: 0 auto 15px;
    }
    @keyframes spin {
        0% { transform: rotate(0deg); }
        100% { transform: rotate(360deg); }
    }
    .footer {
        background: #f5f5f5;
        padding: 15px 20px;
    }

```

```

        text-align: center;
        font-size: 12px;
        color: #999;
        border-top: 1px solid #e0e0e0;
    }
    .alert {
        padding: 12px 16px;
        border-radius: 6px;
        margin-bottom: 15px;
        font-size: 14px;
    }
    .alert-success {
        background: #d4edda;
        border: 1px solid #c3e6cb;
        color: #155724;
    }
    .alert-error {
        background: #f8d7da;
        border: 1px solid #f5c6cb;
        color: #721c24;
    }
}
</style>
</head>
<body>
    <div class="container">
        <div class="header">
            <h1>📁 File Share</h1>
        </div>
        <div class="breadcrumb" id="breadcrumb"></div>
        <div class="content" id="content">
            <div class="upload-section">
                <div class="upload-area" id="uploadArea">
                    <div class="upload-icon">📁</div>
                    <div class="upload-text">Drag files here or click
to select</div>
                    <div class="upload-subtext">Multiple files
supported</div>
                    <input type="file" id="fileInput" multiple style="display:
none;">
                </div>
                <div id="alerts"></div>
                <div class="loader" id="loader">
                    <div class="spinner"></div>
                    <p>Loading... </p>
                </div>
                <div class="file-grid" id="fileGrid"></div>
            </div>
            <div class="footer">Sharing folder contents</div>
        </div>
    <script>
        let currentPath = "/";
        const icons = {
            folder: "📁",
            image: "🖼️",
            video: "🎥",
            audio: "🔊",
            pdf: "📄",
            zip: "📦",
            code: "📄",

```

```

        text: "",
        file: ""
    };

    const uploadArea = document.getElementById("uploadArea");
    const fileInput = document.getElementById("fileInput");
    const alerts = document.getElementById("alerts");

    uploadArea.addEventListener("click", () => fileInput.click());
    uploadArea.addEventListener("dragover", (e) => {
        e.preventDefault();
        uploadArea.classList.add("dragover");
    });
    uploadArea.addEventListener("dragleave", () => {
        uploadArea.classList.remove("dragover");
    });
    uploadArea.addEventListener("drop", (e) => {
        e.preventDefault();
        uploadArea.classList.remove("dragover");
        handleFiles(e.dataTransfer.files);
    });
    fileInput.addEventListener("change", (e) => {
        handleFiles(e.target.files);
    });

    function handleFiles(files) {
        if (files.length === 0) return;
        const formData = new FormData();
        for (let file of files) {
            formData.append("files", file);
        }
        fetch("/api/upload" + currentPath, {
            method: "POST",
            body: formData
        })
        .then(res => res.json())
        .then(data => {
            fileInput.value = "";
            if (data.success) {
                showAlert("✓ Successfully uploaded " +
data.files.length + " file(s)!", "success");
                loadFiles();
            } else {
                showAlert("× Upload failed: " + data.error,
"error");
            }
        })
        .catch(err => {
            showAlert("× Upload error: " + err.message, "error");
        });
    }

    function showAlert(message, type) {
        const alertDiv = document.createElement("div");
        alertDiv.className = "alert alert-" + type;
        alertDiv.textContent = message;
        alerts.appendChild(alertDiv);
        setTimeout(() => {
            alertDiv.remove();
        }, 5000);
    }

```



```

function getFileIcon(name, isDir) {
    if (isDir) return icons.folder;
    const ext = name.split(".").pop().toLowerCase();
    if (["jpg", "jpeg", "png", "gif", "svg", "webp"].includes(ext))
return icons.image;
    if (["mp4", "avi", "mkv", "mov"].includes(ext)) return
icons.video;
    if (["mp3", "wav", "flac", "m4a"].includes(ext)) return
icons.audio;
    if (ext === "pdf") return icons.pdf;
    if (["zip", "rar", "7z", "tar"].includes(ext)) return icons.zip;
    if (["js", "py", "java", "cpp", "go", "html", "css",
"json"].includes(ext)) return icons.code;
    if (["txt", "md", "doc", "docx"].includes(ext)) return icons.text;
    return icons.file;
}

function formatBytes(bytes) {
    if (bytes === 0) return "0 B";
    const k = 1024;
    const sizes = ["B", "KB", "MB", "GB"];
    const i = Math.floor(Math.log(bytes) / Math.log(k));
    return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + " " +
sizes[i];
}

function loadFiles() {
    const loader = document.getElementById("loader");
    const fileGrid = document.getElementById("fileGrid");

    loader.style.display = "block";
    fileGrid.innerHTML = "";

    fetch("/api/list" + currentPath)
        .then(res => res.json())
        .then(items => {
            loader.style.display = "none";

            if (items === null) items = [];

            if (items.length === 0) {
                fileGrid.innerHTML = '<div class="empty-
state"><div class="empty-state-icon">📁</div><p>Folder is empty</p></div>';
                return;
            }

            items.sort((a, b) => {
                if (a.isDir === b.isDir) return
a.name.localeCompare(b.name);
                return b.isDir - a.isDir;
            });

            items.forEach(item => {
                const fileItem =
document.createElement("div");

                fileItem.className = "file-item";

                if (item.isDir) {
                    fileItem.innerHTML = '<div
class="file-icon">' + getFileIcon(item.name, true) + '</div><div class="file-name">' +

```

```

escapeHtml(item.name) + '</div>';

fileItem.addEventListener("click",
() => {
    currentPath = item.path;
    updateBreadcrumb();
    loadFiles();
});
} else {
    fileItem.innerHTML = '<div
class="file-icon">' + getFileIcon(item.name, false) + '</div><div class="file-name">' +
escapeHtml(item.name) + '</div><div class="file-size">' + formatBytes(item.size) +
'</div>';

    fileItem.href = item.path;
    fileItem.addEventListener("click",
(e) => {
        e.preventDefault();
        window.location.href =
item.path;

    });
}

fileGrid.appendChild(fileItem);
});
})
.catch(err => {
    loader.style.display = "none";
    fileGrid.innerHTML = '<div class="empty-state">
<div class="empty-state-icon">✖</div><p>Error loading files</p></div>';
});
}

function updateBreadcrumb() {
    const breadcrumb = document.getElementById("breadcrumb");
    breadcrumb.innerHTML = "";

    const parts = currentPath.split("/").filter(p => p !== "");

    const home = document.createElement("a");
    home.textContent = "⌂ Home";
    home.addEventListener("click", () => {
        currentPath = "/";
        updateBreadcrumb();
        loadFiles();
    });
    breadcrumb.appendChild(home);

    let path = "";
    parts.forEach(part => {
        path += "/" + part;

        const sep = document.createElement("span");
        sep.textContent = "/";
        breadcrumb.appendChild(sep);

        const link = document.createElement("a");
        link.textContent = part;
        link.addEventListener("click", () => {
            currentPath = path;
            updateBreadcrumb();
            loadFiles();
        });
    });
}

```

```

        breadcrumb.appendChild(link);
    });
}

function escapeHtml(text) {
    const map = {
        "&": "&amp;",
        "<": "&lt;",
        ">": "&gt;",
        "\"": "&quot;",
        "'": "&#039;"
    };
    return text.replace(/([<&quot;']/g, m => map[m]);
}

updateBreadcrumb();
loadFiles();
</script>
</body>
</html>`
}

func main() {
    ex, err := os.Executable()
    if err != nil {
        panic(err)
    }
    SHARE_FOLDER = filepath.Dir(ex)

    localIP := getLocalIP()
    //addr := fmt.Sprintf("%s:%d", localIP, PORT)

    http.HandleFunc("/api/list", listFilesHandler)
    http.HandleFunc("/api/upload", uploadFileHandler)
    http.HandleFunc("/", fileHandler)

    fmt.Printf("File Share Server\n")
    fmt.Printf("=====\n")
    fmt.Printf("Sharing: %s\n", SHARE_FOLDER)
    fmt.Printf("URL: http://%s:%d\n", localIP, PORT)
    fmt.Printf("Press Ctrl+C to stop\n\n")

    http.ListenAndServe(fmt.Sprintf(":%d", PORT), nil)
}

```